Tutorato 4 - Esercizi Extra

Matteo Mazzaretto

10 Novembre 2026



Extra 1: crocette

1. Perché `operator++(int)` di un iteratore restituisce un oggetto **per valore** e non per riferimento?
A) perché restituisce una copia locale
B) perché il postfisso non può modificare `*this`
C) perché un operatore non può restituire un riferimento

2. In `const_iterator` il puntatore interno è `const nodo*`. A cosa serve il `const`?
A) a impedire di modificare il nodo
B) a impedire di spostare l'iteratore con `++`
C) a rendere `begin()` ed `end()` più veloci

Extra 1: soluzione

1 → A. Il postfisso salva una copia `aux`, avanza e ritorna `aux`: è una variabile locale che sparisce a fine funzione, quindi va restituita per valore. Il prefisso invece ritorna `*this` per riferimento, perché l'oggetto sopravvive alla chiamata.

2 → A. `const nodo*` è un puntatore a nodo costante: il puntatore si può spostare (`++`, `--`), ma attraverso di esso il nodo non si può modificare. È questo che rende l'iteratore **costante**.

Extra 2: crocette

1. Si vuole scrivere l'ordine lessicografico di `dList<T>` richiedendo il **minimo** al tipo `T`. Cosa basta?

A) serve `operator<`

B) `operator<` e `operator==`

C) serve `operator<=`

2. Come sono ordinate la lista vuota e una lista non vuota?

A) la lista vuota è minore della non vuota

B) la lista non vuota è minore della vuota

C) non sono confrontabili

Extra 2: soluzione

1 → A. Basta `<`. La versione vista a lezione usa anche `==` per decidere se proseguire, ma non è necessario: si prova `x < y`, poi `y < x`, e se nessuna delle due vale si passa al nodo successivo.

2 → A. La lista vuota è un prefisso proprio di ogni altra lista, quindi è minore (come `"" < "a"` per le stringhe). Due liste vuote sono equivalenti: nessuna è minore dell'altra.

Extra 3: cosa stampa?

Usa la classe `dList` vista a lezione, con `const_iterator`. Un `bool` stampato con `cout` vale `0` o `1`.

```
int main(){
  dList<int> l;
  l.insertBack(2); l.insertFront(1);
  l.insertBack(3); l.insertFront(0);

  dList<int>::const_iterator it = l.end();
  --it;
  cout << *it << " ";
  cout << *(--it) << " ";
  cout << *it-- << " ";
  cout << *it << " ";
  ++it; ++it;
  cout << *it++ << " ";
  cout << (it == l.end());
}
```

Extra 3: soluzione

Output:

```
3 2 2 1 3 1
```

Lista: 0 1 2 3

- `insertFront` e `insertBack` costruiscono la lista `0 1 2 3`
- `--it` da `end()` porta sull'ultimo nodo: `3`
- `*(--it)`: il prefisso decrementa **prima**, quindi stampa `2`
- `*it--`: il postfisso restituisce una copia col valore precedente: stampa `2`, poi `it` arretra su `1`
- `*it` stampa `1`
- Due `++` portano `it` da `1` a `3`
- `*it++` stampa `3` e poi `it` avanza al past-the-end: `it == l.end()` vale `1`

Extra 4: cosa stampa?

```
template <class T>
class Box{
  public:
    T* p;
    Box(const T& t): p(new T(t)){ cout << "C "; }
    Box(const Box& b): p(new T(*b.p)){ cout << "K "; }
    Box& operator=(const Box& b){
      if(this != &b){
        delete p;
        p = new T(*b.p);
      }
      cout << "A ";
      return *this;
    }
    ~Box(){ cout << "D" << *p << " "; delete p; }
};

void g(Box<int> x){ *x.p += 10; cout << "g "; }

int main(){
  Box<int> a(1), c(7);
  Box<int> b = a;
  *b.p = 4;
  c = b;
  *b.p = 5;
  g(c);
  cout << *a.p << " " << *b.p << " " << *c.p << " ";
}
```

Extra 4: soluzione

Output:

```
C C K A K g D14
1 5 4 D5 D4 D1
```

- `a(1)` e `c(7)`: due costruttori, `C C`
- `Box<int> b = a;` è un'**inizializzazione**, non un assegnamento: costruttore di copia, `K`. La copia è profonda: `*b.p = 4` non tocca `a`
- `c = b`: `operator=`, `A`; `c` vale 4
- `*b.p = 5` cambia solo `b`: `c` resta 4
- `g(c)`: passaggio per valore, `K`; `x` vale 14 e si stampa `g`; all'uscita `x` si distrugge: `D14`
- `*a.p`, `*b.p`, `*c.p` valgono 1, 5, 4 e vengono stampati attaccati: `154`
- Alla fine si distrugge in ordine inverso di dichiarazione: `b`, `c`, `a`, cioè `D5 D4 D1`

Extra 5: cosa stampa?

`dList(k, t)` crea una lista di `k` nodi uguali a `t`.
`operator<` è l'ordine lessicografico.

```
int main(){
  dList<int> a(3, 2), b(2, 2), c(1, 3), e;
  b.insertBack(1);
  cout << (a < b) << (b < a) << " ";
  cout << (c < a) << (a < c) << " ";
  cout << (e < a) << (a < e) << (e < e) << " ";

  dList<string> s, t;
  s.insertBack("campana"); s.insertBack("zero");
  t.insertBack("campione");
  cout << (s < t) << (t < s);
}
```

Extra 5: soluzione

Output:

```
01 01 100 10
```

- `a` è `2 2 2` e `b` è `2 2 1`: al terzo elemento `1 < 2`, quindi `b < a` e non `a < b`: `01`
- `c` è `3`: al primo elemento `2 < 3`, quindi `a < c` e non `c < a`: `01`
- La lista vuota è minore di ogni lista non vuota, e non è minore di se stessa: `1 0 0`
- `"campana" < "campione"` (a quinta lettera, `a < i`), quindi `s < t` vale `1` e `t < s` vale `0`. Il secondo elemento di `s` non conta più

Extra 6: cosa stampa?

```cpp
template <class C>
void stampa_inv(const C& c){
  typename C::const_iterator it = c.end();
  while(it != c.begin()){
    --it;
    cout << *it << " ";
  }
}

int main(){
  list<int> l;
  for(int i = 1; i <= 5; i++){
    if(i % 2 == 1) l.push_back(i);
    else l.push_front(i);
  }
  stampa_inv(l);
  cout << "| ";

  vector<char> v(3, 'a');
  v.push_back('z');
  v[1] = 'm';
  stampa_inv(v);
  cout << "| ";

  list<int>::iterator it = l.begin();
  ++it;
  l.insert(it, 9);
  it = l.erase(it);
  cout << *it << " | " << l.size() << " | ";
  for(it = l.begin(); it != l.end(); ++it) cout << *it << " ";
}
```

Extra 6: soluzione

Output:

```
5 3 1 2 4 | z
a m a | 1 | 5
| 4 9 1 3 5
```

- Dispari in coda, pari in testa: `l` è `4 2 1 3 5`
- `stampa_inv` parte da `end()` e decrementa **prima** di leggere, perché `end()` non è dereferenziabile. Vale anche per `vector`
- `v` è `a a a z`, poi `v[1] = 'm'` dà `a m a z`: al contrario `z a m a`
- `++it` punta al `2`. `insert(it, 9)` inserisce **prima** di `it`: `4 9 2 1 3 5`, e `it` punta ancora al `2`
- `erase(it)` toglie il `2` e restituisce l'iteratore al **successivo** (`1`): il vecchio `it` non è più valido, per questo si riassegna
- `*it` vale `1`, `size()` vale `5`, lista finale `4 9 1 3 5`

Extra 7: cosa stampa?

```
int main(){
  dList<int> r(2, 5);
  dList<dList<int> > m(3, r);
  r.insertFront(1);
  m.insertBack(r);

  int tot = 0, righe = 0;
  for(dList<dList<int> >::const_iterator i = m.begin(); i != m.end(); ++i){
    righe++;
    for(dList<int>::const_iterator j = i->begin(); j != i->end(); ++j)
      tot += *j;
    cout << tot << " ";
  }
  cout << righe;
}
```

Extra 7: soluzione

Output:

```
10 20 30 41 4
```

- `r` è `5 5`. `m` contiene 3 **copie** di `r` (il costruttore `dList(k, t)` usa il costruttore di copia di `dList<int>`)
- `r.insertFront(1)`: `r` diventa `1 5 5`, ma le righe già in `m` non cambiano: la memoria non è condivisa
- `m.insertBack(r)` aggiunge una quarta riga `1 5 5`
- `i->begin()`: `operator->` restituisce un puntatore alla riga corrente, e su quel puntatore si applica `->begin()`
- Somme progressive: `10`, `20`, `30`, poi `30 + 11 = 41`. Le righe sono 4

Extra 8: scrivi i metodi

Scrivi le definizioni di `void removeFront()` e `void removeBack()`: rimuovono il primo e l'ultimo elemento in tempo $O(1)$ e liberano il nodo.

```
template <class T>
class dList{
  private:
    class nodo{
      public:
        T info;
        nodo* prev;
        nodo* next;
        nodo(const T& t = T(), nodo* p = 0, nodo* n =
0);
    };
    nodo* first;
    nodo* last;  // lista vuota IFF first == last ==
nullptr
  public:
    void insertFront(const T&);
    void insertBack(const T&);
    void removeFront();
    void removeBack();
    // ... big 3, iteratori, ecc. gia' scritti
};
```

```
int main(){
  dList<int> l;
  for(int i = 1; i <= 4; i++)
l.insertBack(i);
  l.removeFront(); l.removeBack();
  cout << l << "| ";
  l.removeBack(); l.removeFront();
  cout << l << "| ";
  l.removeFront();   // lista vuota
  l.insertBack(7);
  cout << l;
}
// output: 2 3 | | 7
```

Extra 8: soluzione

```
template <class T>
void dList<T>::removeFront(){
  if(!first) return;
  nodo* q = first;
  first = first->next;
  if(first) first->prev = nullptr;
  else last = nullptr;
  delete q;
}

template <class T>
void dList<T>::removeBack(){
  if(!last) return;
  nodo* q = last;
  last = last->prev;
  if(last) last->next = nullptr;
  else first = nullptr;
  delete q;
}
```

- Si salva il nodo da cancellare in `q` **prima** di spostare `first`, altrimenti lo si perde
- Se dopo lo spostamento `first` è `nullptr`, la lista aveva un solo nodo: va azzerato anche `last`, altrimenti resta un puntatore a un nodo cancellato
- Altrimenti il nuovo primo non ha predecessore: `first->prev = nullptr`
- `removeBack` è simmetrica
- Nessun ciclo: costo $O(1)$

Extra 9: scrivi il metodo

Nella stessa classe dell'Extra 8 aggiungi `void reverse();`. Scrivine la definizione: inverte la lista **in loco**, senza creare né cancellare nodi (si possono solo cambiare i puntatori).

```
int main(){
  dList<int> m;
  m.reverse();
  cout << m << "| ";
  m.insertBack(1);
  m.reverse();
  cout << m << "| ";
  m.insertBack(2); m.insertBack(3);
  m.insertFront(0);
  m.reverse();
  cout << m;
  m.insertBack(9); m.insertFront(8);
  cout << "| " << m;
}
// output: | 1 | 3 2 1 0 | 8 3 2 1 0 9
```

Extra 9: soluzione

```
template <class T>
void dList<T>::reverse(){
  nodo* p = first;
  while(p){
    nodo* succ = p->next;
    p->next = p->prev;
    p->prev = succ;
    p = succ;
  }
  nodo* t = first;
  first = last;
  last = t;
}
```

- Per ogni nodo si scambiano `prev` e `next`
- `succ` salva il vecchio `next` **prima** dello scambio: dopo, `p->next` punta indietro e non si potrebbe più avanzare
- Alla fine si scambiano `first` e `last`: se mancasse, gli inserimenti successivi agirebbero sull'estremo sbagliato (si vede nell'ultimo pezzo del `main`)
- Lista vuota e lista di un nodo funzionano senza casi speciali

Extra 10: scrivi le funzioni

Scrivi, come funzioni **esterne alla classe** che usano solo l'interfaccia pubblica (`begin`, `end`, `insertBack`):

- `dList<T> concatena(const dList<T>& a, const dList<T>& b)`: restituisce una **nuova** lista con gli elementi di `a` seguiti da quelli di `b`
- `bool operator==(const dList<T>& a, const dList<T>& b)`: due liste sono uguali se hanno la stessa lunghezza e gli elementi uguali a due a due. Su `T` si può usare solo `==`

```
int main(){
  dList<int> a, b;
  a.insertBack(1); a.insertBack(2);
  b.insertBack(3);

  dList<int> c = concatena(a, b);
  cout << c << "| ";
  a.insertBack(9);
  cout << c << "| ";
  cout << (c == concatena(a, dList<int>()));
  cout << (c == c) << (a == b);
}
// output: 1 2 3 | 1 2 3 | 010
```

Extra 10: soluzione

```
template <class T>
dList<T> concatena(const dList<T>& a,
                   const dList<T>& b){
  dList<T> r;
  for(typename dList<T>::const_iterator
      it = a.begin(); it != a.end(); ++it)
    r.insertBack(*it);
  for(typename dList<T>::const_iterator
      it = b.begin(); it != b.end(); ++it)
    r.insertBack(*it);
  return r;
}

template <class T>
bool operator==(const dList<T>& a,
                const dList<T>& b){
  typename dList<T>::const_iterator
    i = a.begin(), j = b.begin();
  for(; i != a.end() && j != b.end();
      ++i, ++j)
    if(!(*i == *j)) return false;
  return i == a.end() && j == b.end();
}
```

- `typename` è obbligatorio: `dList<T>::const_iterator` è un nome dipendente che denota un tipo
- `concatena` ritorna per valore: usa il costruttore di copia (copia profonda), quindi `c` non è legato ad `a` e `b`
- `operator==` si ferma al primo elemento diverso. Alla fine **entrambi** gli iteratori devono essere a `end()`, altrimenti le lunghezze sono diverse
- `!(*i == *j)` richiede solo `==`, non `!=`